ACMEMobility

Progetto di Architetture Software a Microservizi

A.A. 2025/2026

Mattia Albertini – 0001235513

Pierre Errico – 0001235580

28 luglio 2026

Indice

1 Coreografia e sistema proiettato **1**

2 Il diagramma documentativo BPMN **4**

2.1 Applicazione 4

2.2 Servizio stazioni 5

2.3 Servizio Fleet 5

2.4 Servizio bancario 6

2.5 Servizio ACMEMobility 6

2.6 Corrispondenza fra modello e implementazione 7

3 Il diagramma SOA con UML **9**

3.1 Decomposizione dei servizi 9

3.2 Dipendenze e criteri progettuali 10

4 Implementazione **12**

4.1 Servizio bancario 12

4.2 Servizio stazioni 13

4.3 Servizio Fleet management 13

4.4 Servizio ACMEMobility 14

4.5 Servizio di autenticazione 16

4.6 Servizio Reservation 16

4.7 Servizio Logistics 16

4.8 Simulazione dei veicoli 17
4.9 Applicazione 17
4.10 Container e Kubernetes 18
4.11 Gestione dei fallimenti 18
4.12 Verifica 19

5 Conclusioni **21**

Capitolo 1

Coreografia e sistema proiettato

Durante questa prima fase abbiamo analizzato il testo e abbiamo iniziato a suddividerlo per capire quali dovessero essere i singoli elementi della coreografia, ottenendo App, ACME, Banca, Fleet e Stazione. Successivamente abbiamo scritto la coreografia associata al testo basandoci su questi servizi.

ACMEMobility è una piattaforma di vehicle sharing. L'utente può consultare stazioni e veicoli, prenotare un mezzo con un anticipo massimo di 30 minuti oppure iniziare un noleggio immediato mediante QR code, seguirne lo spostamento e riconsegnarlo alla stazione assegnata. La cancellazione è gratuita fino a cinque minuti prima dell'inizio; la cancellazione tardiva e il mancato ritiro entro i cinque minuti di tolleranza producono un addebito di 10 euro. Al ritiro viene bloccata una cauzione di 10 euro. Il costo finale dipende da durata e distanza e aumenta del 10% se la batteria residua è inferiore al 15%.

Indichiamo con $a \to b : m$ l'invio del messaggio m da a a b. La coreografia seguente descrive i due cammini principali; le risposte di errore sono omesse per leggibilità.

```
C ::= (
  App → ACME : lista
  ;
  (
    App → ACME : ritiroQR
    +
    App → ACME : prenota
    ;
```

```
    (App → ACME : ritira
     + App → ACME : annulla
     + ACME : timeout)
  )
  ;
  ACME → Banca : cauzione
  ;
  ACME → Stazione : sblocca
  ;
  ACME → Fleet : avvia
  ;
  (ACME → Fleet : stato)*
  ;
  App → ACME : riconsegna
  ;
  ACME → Fleet : arresta
  ;
  ACME → Stazione : blocca
  ;
  ACME → Banca : addebita
  ;
  ACME → Banca : rilasciaCauzione
  ;
  ACME → App : esito
)*
```

La scelta esterna distingue il ritiro immediato dalla prenotazione. Il ramo di annullamento può terminare senza costo oppure con una penale; il timeout produce la medesima penale. Nel cammino nominale il monitoraggio di Fleet è iterativo, mentre la riconsegna chiude il viaggio soltanto quando il veicolo ha raggiunto la destinazione.

Dopo aver scritto la coreografia, l'abbiamo riscritta come sistema proiettato, riorganizzando i servizi come ruoli. La proiezione assegna ad ACME tutte le decisioni che coinvolgono più domini. App invia comandi e riceve aggiornamenti; Stazione e Banca espongono protocolli conversazionali con stato di sessione; Fleet avvia, interroga e arresta la simulazione. In forma compatta:

```
App ::= lista! ; (ritiroQR! + prenota! ; (ritira! + annulla!))
```

```
          ; (aggiornamento?)* ; riconsegna! ; esito?
ACME ::= lista? ; scelta? ; coordinaRitiro
          ; (statoFleet! ; statoFleet?)* ; riconsegna?
          ; coordinaChiusura ; esito!
Stazione ::= login? ; (sblocca? ; esito! + blocca? ; esito!)* ; logout?
Banca ::= login? ; (cauzione? + addebita? + rilascia?)* ; logout?
Fleet ::= (avvia? ; (stato? ; stato!)* ; arresta? ; riepilogo!)*
```

Le interazioni REST sono sincrone quando serve una risposta immediata; Kafka disaccoppia produzione e lettura della telemetria; gli eventi SSE aggiornano le interfacce senza polling applicativo. Camunda conserva lo stato del processo di noleggio e correla messaggi asincroni mediante gli identificativi di prenotazione e processo.

Questa combinazione deriva dai diversi requisiti delle operazioni. La consultazione di una stazione o la validazione di un QR code richiedono un esito immediato e sono quindi modellate come richieste REST. La telemetria, invece, viene prodotta con continuità e non deve rallentare il simulatore: viene perciò inviata su Kafka e consumata successivamente da Query. I messaggi Camunda rappresentano eventi di dominio che fanno avanzare un'istanza persistente del processo. L'identificativo della prenotazione costituisce la chiave di correlazione e impedisce che un evento relativo a un noleggio modifichi lo stato di un altro.

Capitolo 2

Il diagramma documentativo BPMN

Durante questa fase abbiamo ripreso il sistema proiettato e l'abbiamo riscritto come diagramma BPMN, aggiungendo tutti i calcoli e i controlli dei vari servizi che non potevano essere espressi nella coreografia. Alcune comunicazioni sono state inoltre modificate per rendere più semplice l'implementazione. Il BPMN documentativo è suddiviso in cinque pool, una per ogni ruolo individuato nell'analisi.

I due modelli sono contenuti nei file:

`docs/orchestration/modello_documentativo.bpmn`
`docs/orchestration/processes/process.bpmn`

Il primo descrive le responsabilità organizzative; il secondo traduce la pool ACME in un processo Camunda 8, sostituendo le interazioni astratte con job worker, connettori e messaggi correlati.

2.1 Applicazione

La pool App rimane in attesa di un messaggio da parte di ACME per ottenere la lista delle stazioni e dei veicoli. Per gestire le due scelte dell'utente abbiamo applicato un gateway esclusivo: visualizzare la mappa e avviare un ritiro tramite QR code oppure prenotare un veicolo.

Nel ramo della mappa l'utente visualizza i veicoli e ne sceglie uno; queste azioni sono rappresentate mediante user task. Successivamente invia ad ACME il QR code acquisito. Nel ramo della prenotazione, dopo aver inviato la richiesta, l'utente si trova di fronte a tre possibilità:

- continuare con la prenotazione e notificare ACME al momento del ritiro;
- non ritirare il veicolo, lasciando che il timer di tolleranza scada;
- inviare una richiesta di annullamento.

I due rami che conducono al ritiro si ricongiungono nell'attesa della conferma di ACME. Terminato l'uso del veicolo, l'applicazione invia la richiesta di riconsegna e attende l'esito. Se la destinazione non è ancora stata raggiunta, l'esito `VEHICLE_NOT_AT_DESTINATION` mantiene il processo attivo e consente un nuovo tentativo.

2.2 Servizio stazioni

La pool Stazione rimane in attesa di una notifica tramite un gateway basato su eventi. Le due scelte sono la richiesta di blocco o quella di sblocco di un veicolo. In entrambi i casi l'azione fisica è rappresentata nel modello documentativo da una manual task; al termine viene notificato ad ACME l'esito dell'operazione.

Nell'implementazione il servizio mantiene una sessione SOAP Jolie per ciascuna stazione e offre inoltre un catalogo REST. In questo modo il comando sul blocco viene separato dalla consultazione delle informazioni statiche.

2.3 Servizio Fleet

La pool Fleet rimane in attesa di un messaggio di avvio da parte di ACME, al quale risponde notificandone la ricezione. Successivamente entra in un ciclo di richieste dello stato del veicolo: ricerca e lettura dei dati sono rappresentate mediante service task e l'esito viene inviato ad ACME. Quando arriva la richiesta di arresto, Fleet interrompe la simulazione e restituisce il riepilogo del viaggio.

Nel BPMN eseguibile un sottoprocesso parallelo effettua il monitoraggio ogni due secondi e un evento di confine ne interrompe l'esecuzione alla riconsegna. Posizione e batteria sono

letture eventualmente consistenti, mentre tempo e distanza calcolati dal simulatore sono i valori autorevoli per l'addebito.

2.4 Servizio bancario

La pool Banca rimane in attesa di un messaggio tramite un gateway basato su eventi. Il messaggio può riguardare:

- un addebito, usato in caso di cancellazione tardiva o mancato ritiro;
- una cauzione, bloccata all'inizio del noleggio;
- il rilascio della cauzione, eseguito al termine del pagamento.

Ogni interazione con il conto è rappresentata mediante una service task e il risultato viene notificato ad ACME. Se il saldo non è sufficiente, l'applicazione richiede una ricarica e correla il relativo messaggio al processo in attesa. Il servizio è volutamente dimostrativo e non rappresenta un sistema di pagamento conforme ai requisiti di un sistema bancario reale.

2.5 Servizio ACMEMobility

La pool ACME era stata inizialmente suddivisa in due lane, per differenziare la parte gestionale dalla parte logistica. La prima coordina il processo di noleggio; la seconda gestisce stato, disponibilità e batteria dei veicoli. In fase di implementazione tali responsabilità sono state separate nei servizi ACMEMobility, Reservation, Logistics e Fleet.

La lane gestionale inizia inviando stazioni e veicoli all'applicazione, quindi rimane in attesa della scelta fra QR code e prenotazione. Nel secondo caso un gateway esclusivo gestisce tre continuazioni: ritiro, annullamento o scadenza del tempo disponibile. I controlli temporali sono realizzati mediante timer event. Quando avviene il ritiro, ACME richiede lo sblocco alla stazione, blocca la cauzione e avvia Fleet, attendendo l'esito di ogni servizio.

Durante l'uso del veicolo viene richiesto periodicamente lo stato a Fleet. La richiesta di riconsegna interrompe tale ciclo; ACME verifica la destinazione, arresta Fleet, blocca il veicolo, calcola ed esegue il pagamento, rilascia la cauzione e notifica l'applicazione.

Nell'implementazione ACMEMobility costituisce l'API pubblica e il confine di sicurezza. Valida il JWT, verifica la proprietà delle prenotazioni, inoltra le richieste ai servizi interni e pubblica i messaggi Camunda. Il processo gestisce:

- creazione, annullamento e scadenza della prenotazione;
- cauzione, ricarica eventuale, addebito e rilascio;
- sblocco, avvio Fleet, monitoraggio, blocco e arresto;
- calcolo del prezzo e notifiche SSE verso app e visualizzatore.

Timer e message boundary event modellano esplicitamente finestre temporali, attese e interruzioni, evitando che tali regole restino nel frontend.

Il processo eseguibile realizza quindi una saga orchestrata. Non esiste una transazione distribuita che comprenda Reservation, Logistics, Banca, Stazione e Fleet: ogni partecipante completa una propria operazione locale, mentre Camunda ne registra l'ordine e decide il passo successivo. Gli esiti negativi restano espliciti nel flusso. Per esempio, un saldo insufficiente porta all'attesa della ricarica e una riconsegna lontana dalla destinazione non chiude la prenotazione. Questa soluzione rende visibile lo stato intermedio del noleggio e consente di riprendere l'orchestrazione dopo un errore temporaneo.

2.6 Corrispondenza fra modello e implementazione

La Tabella 2.1 riassume come gli elementi del diagramma documentativo sono stati tradotti nel processo eseguibile. La traduzione non è puramente sintattica: alcune attività astratte sono state scomposte per rendere espliciti correlazione, attese e gestione degli esiti.

Elemento	Implementazione Camunda	Motivazione
Banca e Stazione	Connettore SOAP	Riutilizzo delle interfacce WSDL dei servizi Jolie.
Richiesta HTTP	Connettore REST o client ACMEMobility	Invocazione sincrona con esito disponibile nel processo.
Calcolo del costo	Worker `calculate-charge`	Logica Java testabile, separata dalle espressioni BPMN.
Monitoraggio Fleet	Sottoprocesso con timer di due secondi	Aggiornamenti periodici senza bloccare il flusso principale.
Riconsegna	Messaggio correlato e boundary event	Interruzione del monitoraggio della stessa prenotazione.
Attesa ricarica	Message catch event	Sospensione persistente senza occupare un thread.
Scadenza	Timer event	Applicazione deterministica delle finestre temporali.
Notifica	REST verso ACMEMobility, poi SSE	Disaccoppiamento fra processo e connessione del browser.

Tabella 2.1: Corrispondenza fra BPMN documentativo e processo eseguibile.

Capitolo 3

Il diagramma SOA con UML

3.1 Decomposizione dei servizi

In seguito abbiamo definito la struttura del sistema in un diagramma SOA tramite UML, utilizzando il profilo TinySOA. Ogni servizio è stato definito mediante un package, al quale è stato assegnato uno dei seguenti profili:

- *Task*, se il servizio implementa la logica del sistema;
- *Entity*, se il servizio gestisce un'entità del dominio;
- *Utility*, se il servizio è esterno alla logica applicativa.

Ogni package viene implementato da un'interfaccia con profilo *ServiceInterface* e da una classe con profilo *Capability*. I due elementi sono collegati da una dipendenza *Expose*, diretta dall'interfaccia alla classe. I servizi sono poi collegati mediante dipendenze che partono dalla capability del consumatore e terminano all'interfaccia del fornitore.

Oltre ai servizi già presenti nel diagramma BPMN, per semplificare l'implementazione è stato necessario introdurre nuovi servizi oppure suddividere un servizio in componenti più piccoli. In particolare sono stati aggiunti il simulatore dei veicoli, utile a gestirne il movimento, il servizio di autenticazione, il servizio Reservation e i componenti dedicati alla telemetria. La decomposizione finale è riportata in Tabella 3.1.

Componente	Tipo	Responsabilità principale
ACMEMobility	Task	API pubblica, autorizzazione, correlazione Camunda e SSE
Auth	Entity	utenti, credenziali, recupero password e JWT
Reservation	Entity	ciclo di vita e proprietà delle prenotazioni
Logistics	Entity	veicoli, disponibilità, QR e metadati batteria
Stations	Entity	catalogo stazioni e sessioni di blocco/sblocco
Banking	Task	saldo dimostrativo, cauzione, addebito e ricarica
Fleet	Task	coordinamento del viaggio e verifica della destinazione
Simulator	Task	percorso, movimento, durata, distanza e batteria finale
Tracking/Battery	Task	pubblicazione della telemetria su Kafka
Query	Entity	read model di posizione e batteria
Charge worker	Task	calcolo dell'importo corrente e finale
Camunda	Utility	stato e orchestrazione del processo BPMN
Kafka	Utility	trasporto asincrono della telemetria
GraphHopper	Utility	calcolo dei percorsi sulla mappa di Bologna

Tabella 3.1: Servizi e responsabilità.

3.2 Dipendenze e criteri progettuali

Lo schema definisce App come consumatore di ACMEMobility. Quest'ultimo dipende da Auth, Banking, Reservation, Logistics e Fleet, oltre a Camunda per l'orchestrazione. Reservation gestisce la proprietà e il ciclo di vita delle prenotazioni, mentre Logistics gestisce i veicoli. Fleet è a sua volta dipendente da Query, Simulator, Reservation, Logistics e Stations. Simulator usa GraphHopper e invia i dati ai servizi Tracking e Battery; questi pubblicano su Kafka e Query ne costruisce il modello di lettura.

La separazione tra Reservation e Logistics impedisce che la disponibilità del veicolo coincida con la proprietà della prenotazione. La scomposizione Fleet–Simulator isola la simulazione dalla coordinazione. Tracking e Battery scrivono eventi sui topic `vehicle-tracking` e `vehicle-battery`; Query li combina in un read model, applicando un semplice schema CQRS. GraphHopper e Kafka sono modellati come Utility perché non sono servizi ideati nel progetto, ma dipendenze necessarie alla comprensione dell'architettura. Il charge worker è un

job worker Camunda indipendente e può evolvere senza modificare il diagramma delle API.

La proprietà dei dati segue gli stessi confini. Auth è responsabile degli utenti; Reservation delle prenotazioni; Logistics dell'inventario dei veicoli; Stations del catalogo delle stazioni. Nessun servizio modifica direttamente le tabelle di un altro dominio: le variazioni passano attraverso le interfacce esposte. Il read model di Query non sostituisce le sorgenti autorevoli, ma conserva una proiezione ricostruibile degli eventi di posizione e batteria. La scelta riduce l'accoppiamento sullo schema e consente ai servizi di evolvere in modo indipendente.

Capitolo 4

Implementazione

4.1 Servizio bancario

Il servizio bancario è stato implementato con Jolie, impostando una *inputPort* con protocollo SOAP. Nel server sono state definite le seguenti operazioni:

- `login` e `logout`, per creare e terminare l'ambiente privato dell'utente;
- `charge`, per sottrarre l'importo richiesto dal saldo;
- `holdDeposit`, per bloccare la cauzione e memorizzarla nella sessione privata;
- `releaseDeposit`, per restituire la cauzione;
- `recharge`, per aumentare il saldo quando l'utente autorizza una ricarica.

Il server rimane in attesa di un login e, dopo l'arrivo della richiesta, crea una sessione personale per l'utente. All'interno della sessione rimane in attesa delle operazioni sul conto fino alla ricezione del logout. Rispetto al modello iniziale abbiamo corretto la gestione del saldo insufficiente: il servizio non effettua autonomamente una ricarica senza autorizzazione, ma restituisce l'esito al processo. ACMEMobility notifica l'applicazione e Camunda attende un messaggio di ricarica correlato alla prenotazione.

L'interfaccia è descritta nel file `BankingService.wsdl`, mentre `banking-interface.iol` contiene i tipi e le operazioni condivise. Il file `banking-service.ol` implementa il comportamento conversazionale. Nel deployment Kubernetes indirizzo, porte e parametri iniziali sono passati al container tramite configurazione, evitando di codificare nel processo gli indirizzi del

cluster. Il servizio rimane dimostrativo: saldo e credenziali non hanno le garanzie di sicurezza di un vero intermediario finanziario.

4.2 Servizio stazioni

Il servizio stazioni è stato implementato con Jolie, come il servizio bancario. Utilizza due interfacce differenti: una SOAP per comunicare con il processo Camunda e una REST per fornire a Logistics il catalogo delle stazioni. Nel server sono state definite le funzioni:

- `openSession` e `closeSession`, per creare e terminare le sessioni delle singole stazioni;
- `unlockVehicle` e `lockVehicle`, per rappresentare lo sblocco e il blocco di un veicolo;
- `listStations`, che restituisce l'elenco delle stazioni;
- `getStation`, che restituisce la stazione richiesta.

Il server rimane in attesa dell'apertura di una sessione, della richiesta di una stazione o della richiesta dell'intera lista. Quando arriva una `openSession` viene aperta una sessione privata, nella quale possono essere ricevuti più comandi di blocco e sblocco. La `closeSession` termina la sessione e riporta il servizio nello stato iniziale. Il catalogo e le sessioni delle stazioni, incluso l'identificativo dell'ultimo veicolo gestito, sono persistiti in PostgreSQL tramite il driver JDBC di Jolie; questo evita che la perdita del container comporti la perdita dei dati.

La separazione tra SOAP e REST riflette due tipi di interazione differenti. Il comando di blocco appartiene a una conversazione con stato, mentre la lettura del catalogo è una richiesta priva di stato e può essere esposta come risorsa. In questo modo Logistics non dipende dalla sessione usata da Camunda.

4.3 Servizio Fleet management

Come anticipato nel diagramma UML, il servizio Fleet management è suddiviso nei componenti Fleet, Simulator, Tracking, Battery e Query. Tutti questi componenti sono implementati in Java 21 con Helidon MicroProfile 4.3.4.

Il `tracking-service` riceve dal simulatore la posizione del veicolo e la pubblica sul topic Kafka `vehicle-tracking`. In modo analogo, il `battery-service` pubblica la percentuale

di carica sul topic `vehicle-battery`. I producer richiedono conferma da tutte le repliche con `acks=all`, abilitano l'idempotenza e configurano retry e timeout. Queste impostazioni riducono il rischio di perdita o duplicazione degli eventi durante errori transitori.

Il `query-service` consuma entrambi i topic e mantiene l'ultima fotografia disponibile per ciascun veicolo. In questo modo viene realizzato un semplice modello CQRS: Tracking e Battery costituiscono il lato di scrittura degli eventi, mentre Query costituisce il lato di lettura. Le due misure possono arrivare in istanti differenti; di conseguenza posizione e batteria mostrate nell'interfaccia sono eventualmente consistenti.

Ogni evento contiene l'identificativo del veicolo e il valore misurato. Query usa tale identificativo come chiave logica e aggiorna soltanto la parte del modello interessata dall'evento ricevuto. Un aggiornamento di posizione non sovrascrive quindi la batteria e viceversa. La configurazione idempotente dei producer riduce i duplicati dovuti ai retry; il consumatore, memorizzando l'ultima fotografia, rende comunque innocua la ripetizione dello stesso valore. Questa consistenza eventuale è accettabile per la visualizzazione, ma non per il calcolo economico: minuti, chilometri e batteria finale vengono richiesti direttamente al simulatore al termine del viaggio.

Il `fleet-service` coordina il sottosistema. Per iniziare un viaggio recupera i dati della prenotazione e del veicolo, seleziona la destinazione e richiede al simulatore di iniziare il percorso. Durante il viaggio interroga Query e Simulator per produrre lo stato richiesto dal BPMN. Alla riconsegna controlla che il veicolo abbia raggiunto la stazione assegnata: in caso contrario restituisce l'esito `VEHICLE_NOT_AT_DESTINATION`; in caso positivo arresta la simulazione e restituisce minuti, chilometri e batteria finale.

4.4 Servizio ACMEMobility

La parte gestionale di ACME è stata realizzata come BPMS tramite Camunda 8. Per farlo abbiamo ripreso il BPMN documentativo, isolando la parte gestionale e trasformandola in un processo eseguibile. Le coppie di task che comunicano con i servizi Jolie sono state sostituite da connettori SOAP; le comunicazioni con API HTTP sono state sostituite da connettori REST o da job worker. Le variabili del processo contengono identificativi, tempi e risultati necessari alla correlazione, mentre indirizzi e credenziali provengono dalla configurazione del

deployment.

Il ciclo di lettura dello stato del veicolo è stato spostato in un sottoprocesso parallelo, così da poterlo gestire indipendentemente dal flusso principale. Il ciclo viene attivato da un timer ogni due secondi e termina mediante un evento di confine quando arriva la richiesta di riconsegna. Questa soluzione sostituisce il gateway basato su eventi del diagramma documentativo e permette al processo di continuare a ricevere messaggi mentre il monitoraggio è in corso.

ACMEMobility è inoltre un microservizio Helidon che funge da API pubblica. Espone le operazioni di autenticazione, stazioni, prenotazioni, QR, banca e Fleet; valida il JWT, ricava l'identità dell'utente e impedisce l'accesso alle prenotazioni altrui. I client interni comunicano con i rispettivi microservizi, mentre il client Camunda avvia istanze e pubblica messaggi correlati.

Per notificare il browser sono stati usati Server-Sent Events. Sono presenti flussi distinti per prenotazioni, veicoli e visualizzatore della simulazione. La comunicazione è principalmente server-to-client e non richiede la bidirezionalità di WebSocket; SSE permette inoltre al browser di ristabilire la connessione in modo semplice.

Il calcolo dell'addebito è affidato al job worker `calculate-charge`, scritto in Java. Esso riceve minuti, chilometri e batteria e applica la formula

$$C = 1{,}00 + 0{,}20\,m + 0{,}50\,k,$$

dove m rappresenta i minuti e k i chilometri. Se la batteria è inferiore al 15%, il costo viene moltiplicato per $1{,}10$; l'importo finale è arrotondato ai centesimi. Il worker valida inoltre che minuti e chilometri siano non negativi e che la batteria appartenga all'intervallo $[0, 100]$. Un secondo worker, `stop-and-charge`, gestisce la ricarica di sicurezza: quando la batteria raggiunge la soglia critica, richiede a Fleet di ricaricare il veicolo al 25% e di riprendere la simulazione ancora attiva.

La separazione del calcolo in un worker evita di inserire codice complesso nelle espressioni BPMN e rende la regola tariffaria verificabile tramite test unitari. Il worker restituisce sia l'importo base sia quello finale e indica esplicitamente se la maggiorazione è stata applicata. Il processo può così notificare all'utente il motivo della differenza, mantenendo distinta la decisione economica dall'operazione SOAP che modifica il saldo.

4.5 Servizio di autenticazione

Il servizio di autenticazione è implementato con Helidon e separa la gestione degli utenti dal resto del dominio. Espone registrazione, login, cambio password e recupero password. Le credenziali e i dati dell'utente sono persistiti in PostgreSQL; Flyway crea e aggiorna lo schema in fase di avvio. Per i test è disponibile un repository in memoria, così da non rendere le verifiche unitarie dipendenti dal database.

Dopo un login valido il servizio genera un JWT. Il token viene inviato dal frontend nell'header `Authorization` e verificato da ACMEMobility prima di inoltrare una richiesta. In questo modo i servizi interni non sono esposti direttamente al browser. Il recupero password usa un'astrazione `PasswordMailer`, con implementazione SMTP e sostituto controllabile nei test. Le configurazioni incluse nel repository sono adatte alla dimostrazione, ma chiavi, password e segreti dovrebbero essere ruotati e gestiti esternamente in un ambiente produttivo.

4.6 Servizio Reservation

Reservation gestisce la proprietà e il ciclo di vita della prenotazione. I dati principali sono identificativo, utente, veicolo, stazioni, orario previsto e stato. La persistenza usa PostgreSQL, pool HikariCP e migrazioni Flyway. Le transizioni sono esposte sia mediante API REST interne sia mediante job worker Camunda, in modo che il processo possa creare, ritirare, annullare, far scadere e completare una prenotazione.

La prenotazione può iniziare nell'intervallo compreso tra l'istante corrente e i 30 minuti successivi. L'annullamento è gratuito fino a cinque minuti prima dell'orario stabilito; dopo tale soglia viene applicata la penale di 10 euro. Anche il mancato ritiro entro i cinque minuti di tolleranza produce lo stesso addebito. Il servizio associa sempre la prenotazione all'utente autenticato; ACMEMobility verifica tale proprietà prima di pubblicare comandi Camunda.

4.7 Servizio Logistics

Logistics gestisce inventario e stato operativo dei veicoli: stazione, disponibilità, batteria, coordinate e QR code. I dati sono memorizzati in PostgreSQL e inizializzati tramite migrazioni

Flyway. Il servizio consulta il catalogo REST di Stations per associare correttamente i mezzi alle stazioni.

Quando viene creata una prenotazione, Logistics pone il veicolo in stato riservato; al ritiro lo rende in uso; al termine aggiorna stazione, batteria e disponibilità. La separazione da Reservation evita che i dati relativi alla proprietà del noleggio vengano confusi con quelli dell'inventario. Nel flusso QR, ACMEMobility confronta il codice inserito con il valore restituito da Logistics prima di avviare il processo.

4.8 Simulazione dei veicoli

Il simulatore è un servizio Helidon che riproduce lo spostamento del veicolo. All'avvio riceve posizione iniziale e destinazione, quindi usa GraphHopper e la mappa di Bologna inclusa nel progetto per calcolare un percorso stradale. Il movimento viene avanzato per passi; a ogni aggiornamento il simulatore invia la posizione a Tracking e la carica a Battery.

Il simulatore mantiene i valori autorevoli di tempo trascorso e distanza percorsa. Fleet li usa sia durante il monitoraggio sia nel riepilogo finale, evitando che il costo dipenda dal numero di richieste del frontend. Quando la batteria raggiunge una soglia critica il processo può fermare il veicolo, richiedere una ricarica e riprendere la simulazione. Il visualizzatore mostra separatamente percorso, posizione, batteria, durata, chilometri e importo corrente.

4.9 Applicazione

L'applicazione è stata sviluppata come single-page application mediante React, TypeScript, HTML e CSS, con Vite come strumento di build. Le viste gestiscono login, registrazione, mappa, elenco delle prenotazioni, ritiro, annullamento e riconsegna. L'utente può selezionare una stazione e un veicolo, indicare l'orario di inizio entro la finestra ammessa oppure inserire il QR code per il ritiro immediato.

Le notifiche SSE aggiornano l'interfaccia senza polling continuo e mostrano conferma, cauzione, eventuale penale, stato del viaggio e pagamento finale. Una seconda applicazione React, il simulation visualizer, combina la posizione e la batteria lette da Query con durata, chilometri

e costo inviati da ACMEMobility. La distinzione evita di accoppiare la UI operativa a quella di osservazione.

4.10 Container e Kubernetes

Ogni servizio dispone di un Dockerfile. Gli script nella directory `scripts` preparano un profilo Minikube, creano i namespace `acme`, `orchestration` e `kafka`, installano Camunda tramite Helm e Kafka tramite Strimzi, costruiscono le immagini all'interno di Minikube e attendono la readiness dei deployment.

Kafka usa i topic `vehicle-tracking` e `vehicle-battery`; PostgreSQL è distribuito nel namespace applicativo; GraphHopper usa la mappa inclusa nel repository. L'API ACMEMobility e il visualizzatore sono esposti rispettivamente sulle NodePort 30080 e 30081, mentre i servizi interni rimangono raggiungibili soltanto nel cluster. Secret ed environment variable separano credenziali e configurazione dalle immagini.

I servizi applicativi sono raggiunti mediante nomi DNS interni stabili, indipendenti dall'indirizzo dei singoli pod. Le due NodePort costituiscono deliberatamente l'unico ingresso dall'host: il browser comunica con ACMEMobility e con il visualizzatore, mentre database, broker e API interne non vengono pubblicati all'esterno. Gli script attendono inoltre il rollout dei deployment prima di avviare le verifiche, evitando che una dipendenza ancora in fase di inizializzazione venga interpretata come errore applicativo.

La procedura completa richiede risorse elevate soprattutto a causa di Camunda, Elasticsearch, Kafka e GraphHopper. La configurazione proposta assegna 16 GiB di memoria e 8 CPU al profilo Minikube. Gli script non eliminano automaticamente un profilo esistente con risorse insufficienti, per evitare la perdita accidentale dello stato dell'utente.

4.11 Gestione dei fallimenti

L'orchestrazione riduce il rischio che un errore temporaneo renda incomprensibile lo stato del noleggio, ma non equivale a una transazione distribuita. Se un servizio non risponde, l'istanza Camunda conserva le variabili e il punto di avanzamento; il job o il connettore può essere

ritentato senza ricostruire il flusso dal browser. I comandi sono correlati alla prenotazione e le verifiche di stato nei servizi limitano l'effetto di messaggi ripetuti.

Rimangono casi che, in un sistema produttivo, richiederebbero compensazioni più articolate. Se la cauzione viene bloccata ma lo sblocco fallisce definitivamente, il processo dovrebbe rilasciarla esplicitamente. Se Fleet si arresta dopo l'avvio, occorre ricostruire lo stato dal simulatore prima di accettare la riconsegna. Un ritardo di Kafka può rendere temporaneamente obsoleta la posizione mostrata, ma non modifica l'addebito, che usa il riepilogo autorevole del simulatore. Per completare l'industrializzazione sarebbero inoltre utili dead-letter queue, politiche di retry differenziate, timeout per ogni dipendenza e operazioni amministrative per correggere un'istanza bloccata.

4.12 Verifica

La verifica combina test automatici e scenari end-to-end. I test JUnit coprono le risorse Helidon, le transizioni principali e il calcolo dell'addebito. Per i frontend la verifica automatica comprende la compilazione TypeScript, la build di produzione e l'analisi statica con ESLint; non sono presenti test automatici dei componenti React. Le procedure manuali di smoke test descritte nel README verificano autenticazione, prenotazione, ritiro, telemetria, riconsegna, cancellazione e ricarica.

Le prove sono organizzate su livelli differenti. I test unitari isolano le regole deterministiche, come formula tariffaria, validazione degli input e comportamento dei repository sostitutivi. I test dei singoli servizi verificano contratti HTTP, codici di risposta e serializzazione. Le procedure di smoke test vengono invece eseguite manualmente sul cluster completo e controllano che Camunda, Jolie, PostgreSQL, Kafka e i servizi Helidon collaborino nello stesso scenario. La prova manuale del frontend completa la verifica osservando notifiche SSE, aggiornamento della mappa e transizioni visibili all'utente.

Questa distinzione è importante perché un test unitario non può dimostrare la corretta correlazione di un messaggio Camunda o la consegna della telemetria attraverso Kafka. Simmetricamente, uno scenario end-to-end è più lento e rende meno immediata l'individuazione di un errore nella formula. L'uso congiunto dei livelli bilancia rapidità diagnostica e copertura dell'integrazione.

I criteri di accettazione principali sono:

- una prenotazione riserva un solo veicolo e rispetta le finestre temporali;
- un QR errato non avvia il processo e un utente non può operare sulle prenotazioni altrui;
- posizione e batteria raggiungono Query attraverso Kafka;
- una richiesta di riconsegna anticipata non chiude il viaggio;
- addebito, maggiorazione e cauzione concordano con il riepilogo finale;
- tutti i deployment applicativi raggiungono lo stato *Ready*.

Capitolo 5

Conclusioni

Il progetto realizza l'intero ciclo di un noleggio mediante un'architettura eterogenea a microservizi. L'orchestrazione BPMN rende esplicite regole, timeout e compensazioni; REST e SOAP gestiscono le richieste con risposta, Kafka disaccoppia la telemetria e SSE aggiorna le interfacce. PostgreSQL assegna la proprietà dei dati ai rispettivi servizi, mentre Kubernetes rende riproducibile il deployment.

Rispetto a un'applicazione monolitica, questa soluzione introduce un costo operativo evidente: più processi, configurazioni, immagini, protocolli e dipendenze devono essere distribuiti e osservati. Una singola operazione di noleggio attraversa più confini di rete e non può affidarsi a una sola transazione ACID. In compenso, i confini rendono esplicita la proprietà dei dati, consentono di sostituire o scalare separatamente i componenti e permettono di integrare tecnologie eterogenee, come Jolie, Camunda e Helidon, attraverso contratti definiti.

Nel contesto del progetto la decomposizione è volutamente più articolata di quanto richiederebbe il solo carico dimostrativo. Il suo valore risiede nella possibilità di studiare orchestrazione, consistenza eventuale, interoperabilità e deployment cloud-native su un caso d'uso completo. Per un prodotto con scala ridotta, alcuni servizi potrebbero essere inizialmente accorpati e separati soltanto in presenza di esigenze organizzative o di carico misurabili.

Il principale compromesso è la complessità operativa: per un prototipo, il numero di componenti e il consumo di risorse sono superiori a quelli di un'applicazione monolitica. Tale costo è giustificato dall'obiettivo didattico di integrare orchestrazione, messaggistica, protocolli eterogenei e deployment cloud-native. Restano fuori dall'ambito produttivo la gestione

sicura dei segreti, l'alta disponibilità, la cifratura end-to-end, l'osservabilità centralizzata e un vero payment provider. Questi aspetti costituiscono le evoluzioni naturali del sistema.